

Ejercicio Básico para Aplicar el Patrón de Diseño Command

Tema: Sistema de Control Multimedia

1. Planteamiento del problema

Una empresa desea desarrollar una aplicación de escritorio para controlar un reproductor multimedia. La interfaz del sistema tendrá varios botones que permitirán ejecutar acciones sobre el reproductor de música.

Las acciones iniciales del sistema serán:

- Reproducir música
- Pausar música
- Detener reproducción

El problema es que el panel de botones no debe depender directamente del reproductor multimedia, ya que en el futuro podrían agregarse nuevas funcionalidades como:

- Adelantar canción
- Retroceder canción
- Subir o bajar volumen
- Repetir canción

Se requiere implementar una solución utilizando el patrón de diseño Command, de forma que el panel de control pueda ejecutar acciones sin conocer los detalles internos del reproductor.

2. Requisitos funcionales básicos

Requisitos obligatorios:

1. El sistema debe permitir reproducir música.
2. El sistema debe permitir pausar música.
3. El sistema debe permitir detener la reproducción.
4. Cada acción debe estar encapsulada en un comando independiente.
5. El botón de control no debe interactuar directamente con el reproductor multimedia.
6. Debe utilizarse una interfaz común para todos los comandos.
7. El programa debe ejecutarse correctamente desde main().

Requisito opcional (reto extra):

- Implementar un historial de acciones para simular una funcionalidad de Undo/Redo.

3. Clases sugeridas para el Diagrama

Interfaz:

Command

- execute()

Comandos concretos:

- PlayCommand
- PauseCommand
- StopCommand

Clase receptora:

MusicPlayer

Métodos:

- play()
- pause()
- stop()

Clase invocadora:

Button

Método:

- press()

Clase principal:

Main

4. Instrucciones de desarrollo

1. Crear la interfaz Command con el método execute().
2. Crear la clase MusicPlayer, que actuará como Receiver.
3. Crear los comandos concretos: PlayCommand, PauseCommand y StopCommand.
4. Crear la clase Button, que actuará como Invoker.
5. En Main, crear el reproductor, comandos y botones.
6. Ejecutar las acciones simulando el uso de botones.

5. Resultado esperado

Al ejecutar el programa, debería mostrarse una salida similar a:

Reproduciendo música...

Música pausada...

Música detenida...

El sistema debe demostrar que:

- Los botones no conocen la lógica interna del reproductor.
- Cada acción se ejecuta mediante un comando independiente.
- El patrón Command está implementado correctamente.

6. Preguntas de reflexión

1. ¿Qué problema de diseño resuelve el patrón Command en este sistema?
2. ¿Qué sucedería si el botón dependiera directamente de MusicPlayer?
3. ¿Cómo podría agregarse una nueva funcionalidad sin modificar el código existente?
4. ¿Qué ventajas ofrece encapsular las acciones en objetos?
5. ¿Cómo podría implementarse Undo/Redo usando este patrón?
6. ¿Qué principios de diseño orientado a objetos se aplican en esta solución?

Objetivo del ejercicio

El propósito de esta actividad es comprender cómo el patrón Command permite desacoplar el objeto que solicita una acción del objeto que realmente la ejecuta, mejorando la extensibilidad, organización y mantenimiento del software.